CJ.SCADA FTMS · 개발 참고 문서

CJ.SCADA FTMS
데이터 포맷·프로토콜 명세서

에뮬레이터 ↔ app-server (TCP) · app-server ↔ Viewer (REST · SignalR)

대상	tcplinent.emul.console.ftms · app-server.ftms · app-shared.ftms(메시지 모델) · DxfBlazorViewer.Client
기준 소스	CJ.SCADA.Solution.zip 원본 (소스 분석 기준, 패킷 캡처 아님)
검증 환경	Windows · VS2026 · .NET 8 · WSL Docker (PostgreSQL 17 + TimescaleDB, nginx)
관련 문서	01 최종 분석 보고서 · 03 부록 REST API · 04 주요 클래스·함수 설명서
표기	★ 핵심 · ⚠ 주의 · (Px) 본 문서 §7 · (Sx/Bx) 보고서 01 §7

목차

1. 전체 통신 구성
    1.1 구간별 요약
    1.2 데이터 흐름 한 줄 요약
2. TCP 프레임 (에뮬레이터 ↔ 서버)
    2.1 프레임 구조 ★
    2.2 연결과 시간 규칙
3. TCP 메시지 공통 규칙
    3.1 JSON 직렬화 규칙 (Json.cs)
    3.2 type 값과 서버 처리
    3.3 JSON 이 아닌 텍스트 명령 (핸들러 우선순위 순)
4. TCP 메시지별 명세
    4.1 heartbeat — 장치 상태 보고 ★
    4.2 processcount — 처리수량 보고 ★
    4.3 docksortingparam — 분류 파라미터 세트 요청
    4.4 requesttime — 시각 요청
5. 장치 상태 문자열 변환 규칙 ★
    5.1 단계별 변환
    5.2 화면 판정 규칙 (Viewer)
    5.3 ⚠ 에뮬레이터 컨베이어 값은 모두 Stop 으로 표시됨 (P2)
6. 웹 통신 (서버 ↔ Viewer)
    6.1 공통
    6.2 Viewer 가 주기적으로 부르는 API
    6.3 /api/Dock/status 배열형 포맷 ★
    6.4 SignalR /chathub
    6.5 인증 (로그인 화면)
7. 프로토콜 관련 문제점
8. 직접 확인하는 방법
    8.1 TCP 를 손으로 보내 보기 (PowerShell)
    8.2 상태 API 확인
    8.3 서버 로그에서 송수신 보기

부록. 메시지 예제 모음
    A. heartbeat 요청 (에뮬레이터 B22, 실제 직렬화 형태)
    B. heartbeat 응답
    C. 서버 메모리에 저장되는 정규화 결과 (A 를 받은 뒤)
    D. processcount 요청
    E. processcount 응답
    F. docksortingparam 요청 / 응답
    G. /api/Dock/status 응답 (도크 2개)

그림 목록

1. 그림 1. 전체 통신 구성
2. 그림 2. TCP 세션과 메시지 순서
3. 그림 3. 분류 파라미터 전달 경로
4. 그림 4. 장치 상태 문자열 변환 단계
5. 그림 5. Viewer ↔ 서버 통신 순서

1. 전체 통신 구성

1.1 구간별 요약

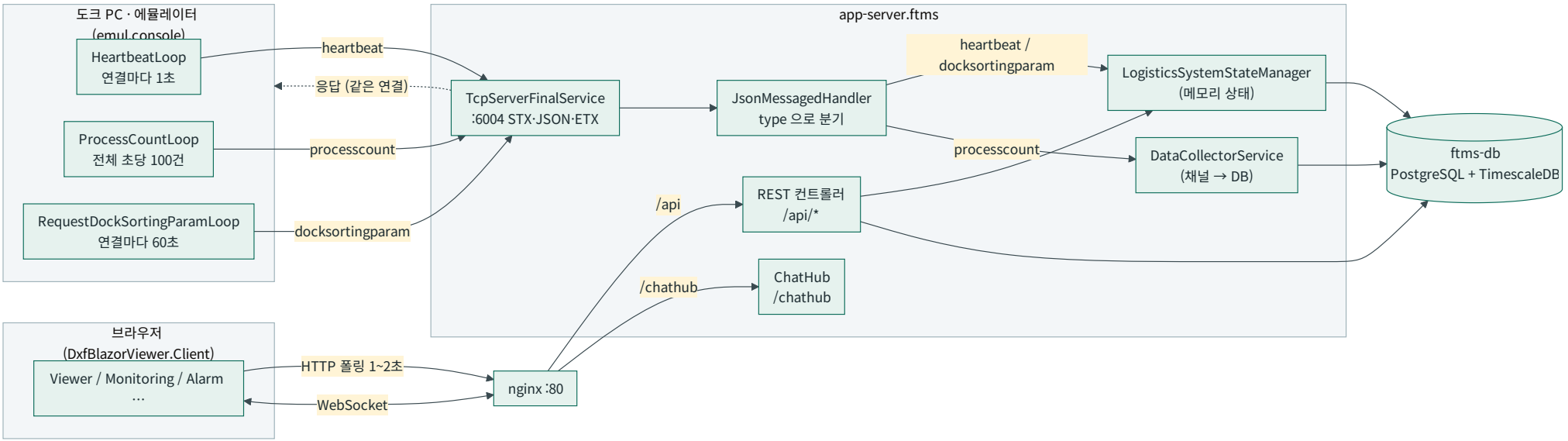
구간	방향	전송 방식	포트 / 경로	데이터 형식
에뮬레이터 → 서버	요청	TCP	6004 (6005, 6006 도 열림)	STX + JSON(UTF-8) + ETX
서버 → 에뮬레이터	응답	같은 TCP 연결	〃	STX + JSON 또는 텍스트 + ETX
Viewer → 서버	요청	HTTP (nginx :80 경유)	`/api/*`	JSON, 도면은 MessagePack + LZ4
서버 → Viewer	응답	〃	〃	JSON(camelCase), `/api/Dock/status` 는 배열형 JSON
서버 → Viewer	푸시	SignalR (WebSocket)	`/chathub`	SignalR JSON 프로토콜

에뮬레이터와 Viewer 는 직접 통신하지 않습니다. 도크가 보낸 데이터는 **서버 메모리(LogisticsSystemStateManager)와 DB 를 거쳐서** 화면에 나타납니다.

1.2 데이터 흐름 한 줄 요약

- **장치 상태**는 이렇게 흐릅니다.
  1. 에뮬레이터가 `heartbeat` 를 1초마다 보냅니다.
  2. 서버가 상태 문자열을 정규화해 메모리에 저장하고, 10초마다 DB 에 기록합니다.
  3. Viewer 가 `/api/Dock/status` 를 1초마다 폴링해 도식에 색으로 표시합니다.
- **처리수량**은 이렇게 흐릅니다.
  1. 에뮬레이터가 `processcount` 를 보냅니다(전체 초당 약 100건).
  2. 서버가 채널에 넣고, 5분 누적은 메모리에, 원본·집계는 DB 에 저장합니다.
  3. Viewer 가 `/api/ProcessCount/live/latest-n-minutes` 를 2초마다 폴링합니다.
- **분류 파라미터**는 이렇게 전달됩니다.
  1. 설정 화면에서 `POST /api/setting/json` 을 보내면 DB 와 서버 메모리가 바뀝니다.
  2. 서버가 다음 `heartbeat` 응답의 `setParameter` 로 도크에 알립니다.
  3. 도크는 `docksortingparam` 요청으로 파라미터 세트 전체를 받습니다.

그림 1. 전체 통신 구성



- 에뮬레이터(도크)는 TCP 6004 로 세 종류의 메시지를 보내고, 같은 연결로 응답을 받습니다.
- 서버는 `type` 값으로 상태 관리(heartbeat, docksortingparam)와 수집(processcount)으로 나눕니다.
- 브라우저는 nginx(:80)를 거쳐 REST 폴링과 SignalR(WebSocket)로 서버와 통신합니다.
- 에뮬레이터와 Viewer 는 직접 통신하지 않고 서버 메모리·DB 를 거칩니다.

2. TCP 프레임 (에뮬레이터 ↔ 서버)

2.1 프레임 구조 ★

```
+------+----------------------------------------+------+

| 0x02 |  payload (UTF-8 text, usually 1 JSON)  | 0x03 |
| STX  |                                        | ETX  |
+------+----------------------------------------+------+
```

항목	값	설정 위치
시작 구분자	`\u0002` (STX, 1바이트)	서버 `TcpServerFinal:StartDelimiter`, 에뮬레이터 `TcpClient:StartDelimiter`
끝 구분자	`\u0003` (ETX, 1바이트)	`EndDelimiter` (양쪽)
인코딩	UTF-8	`Encoding.UTF8` 고정
길이 필드	없음 (구분자로만 나눔)	—
한 프레임	메시지 1개	—
압축·암호화	없음 (평문)	—

- **서버의 수신 처리** (`ProtocolParser.TryReadFrame`):
  1. 버퍼에서 STX 를 찾고, **STX 앞에 있는 바이트는 버립니다.**
  2. STX 뒤에서 ETX 를 찾아 그 사이를 메시지 1개로 꺼냅니다.
  3. ETX 가 아직 오지 않았으면 다음 수신을 기다립니다. 한 번에 여러 프레임이 와도, 프레임 하나가 여러 번에 나뉘어 와도 올바르게 처리됩니다.
- **송신**: 서버와 에뮬레이터 모두 `STX + 본문 + ETX` 를 하나의 배열로 합쳐 `WriteAsync` 한 번에 보냅니다.
- **JSON 안의 제어문자**: System.Text.Json 은 0x02, 0x03 같은 제어문자를 항상 `\u0002` 처럼 이스케이프합니다. 그래서 JSON 본문 안에 **실제 STX/ETX 바이트가 섞일 일은 없습니다.** 컨베이어 상태의 `\u0000` 도 마찬가지로 이스케이프되어 안전합니다.
- ⚠ **최대 길이 제한이 없습니다.** ETX 가 끝내 오지 않으면 버퍼가 계속 커집니다 (P10).
- `StartDelimiter` 를 비워 두면 Line 모드가 되어 ETX(또는 지정 문자열)로만 나눕니다. 서버의 `TcpClientFinal·ExternalServers` 설정에는 `<ETX>`, `[STX]` 같은 문자열 구분자가 쓰입니다. 이 설정들은 다른 외부 연동용이고, 에뮬레이터 구간과는 관계가 없습니다.

2.2 연결과 시간 규칙

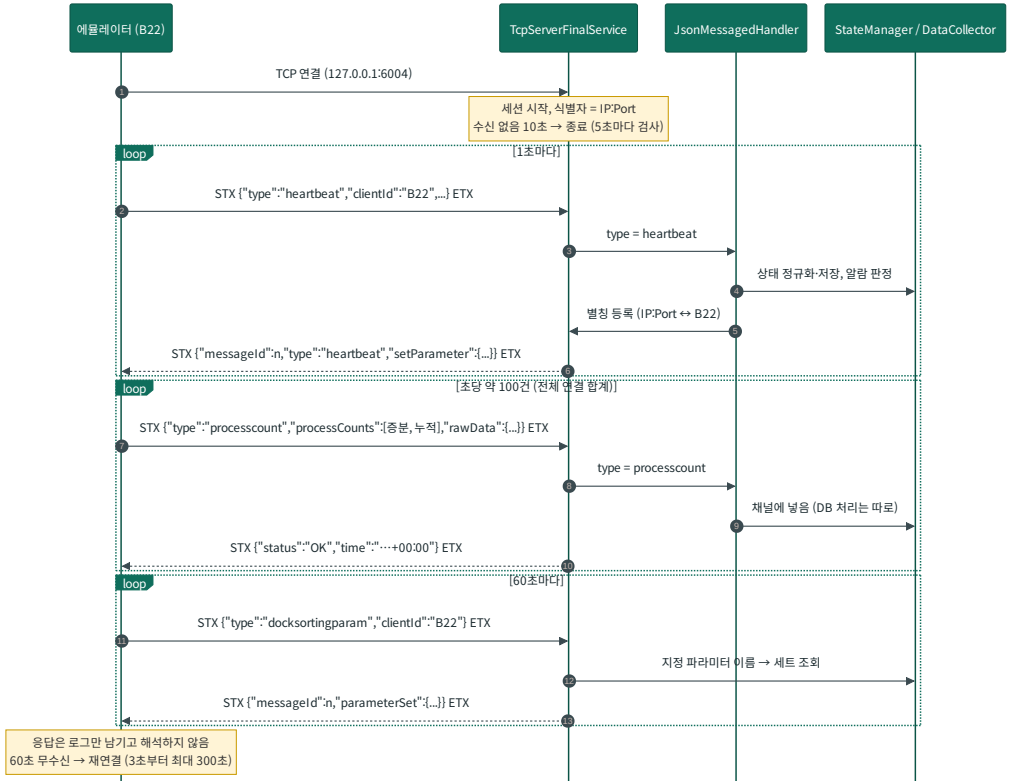
항목	서버 (app-server)	에뮬레이터 (emul.console)
역할	Listen `0.0.0.0:6004/6005/6006`	`Servers[]` 항목마다 연결 1개 (기본 57개, 모두 127.0.0.1:6004)
연결 식별	처음에는 원격 `IP:Port` 로 식별, heartbeat 를 받으면 별칭(nickname)을 `clientId` 로 연결	`ServerId` (예: `B22`)를 그대로 메시지의 `clientId` 로 사용
수신 타임아웃	`TimeoutSeconds` = **10초** 동안 수신이 없으면 연결 종료 (5초마다 검사)	`TimeoutSeconds` = **60초** 동안 수신이 없으면 연결 종료 후 재연결
재연결	—	`ReconnectDelaySeconds` 3초부터 늘려 가며 최대 300초
TCP Keep-Alive	—	사용 (60초 / 10초)

- 에뮬레이터가 시작되면 2초 기다린 뒤 아래 송신 루프 세 개를 돌립니다 (`Worker.StartBroadcast`).

루프	주기	메시지
`HeartbeatLoop`	**연결마다 1초**	`heartbeat`
`ProcessCountLoop`	**전체 초당 100건**을 임의의 연결에 나눠 보내고, 건마다 0~1초 임의 지연	`processcount`
`RequestDockSortingParamLoop`	**연결마다 60초**	`docksortingparam`

- 에뮬레이터는 서버 응답을 **로그로만 남기고 해석하지 않습니다** (`OnMessageReceived`). 응답의 `setParameter` 같은 값을 받아도 아무 동작을 하지 않습니다.

그림 2. TCP 세션과 메시지 순서



- 모든 프레임은 `STX(0x02) + JSON + ETX(0x03)` 입니다.
- heartbeat 응답의 `setParameter` 로 서버가 도크 설정을 지시합니다.
- processcount 응답은 채널에 넣는 즉시 보내며 `messageId` 가 없습니다 (P9).
- 서버는 10초, 에뮬레이터는 60초 무수신이면 연결을 끊습니다 (P12).

3. TCP 메시지 공통 규칙

3.1 JSON 직렬화 규칙 (`Json.cs`)

규칙	송신 (`Json.Serialize`)	수신 (`Json.Deserialize`)
속성 이름	**camelCase** (`MessageId` → `messageId`, `Read_D` → `read_D`)	대소문자 구분 안 함
null 값	**속성 자체를 생략** (`WhenWritingNull`)	없으면 기본값
날짜 (`DateTimeOffset`)	보내는 PC 의 **로컬 시간 + 오프셋**, ISO 8601 (예: `2026-10-20T14:00:00.1234567+09:00`)	문자열을 파싱하며 오프셋은 그대로 유지
숫자	숫자	문자열 숫자(`"12"`)도 허용
기타	한글은 이스케이프하지 않음	주석과 끝 쉼표 허용

- 메시지 종류는 **루트의 `type` 속성 하나로만** 구분합니다. 서버는 `Json.FastSearch` 로 루트에서 `type` 만 빠르게 읽어 핸들러를 고릅니다.
- 공통 머리 (`TcpMessageRequestBase` / `TcpMessageResponseBase`) — 순서는 `JsonPropertyOrder` 로 고정되어 있습니다.

순서	속성	형식	요청	응답	설명
1	`messageId`	long	✔	✔	송신자가 1씩 올리는 번호. 응답은 요청의 번호를 그대로 돌려줌(heartbeat, docksortingparam)
2	`type`	string	✔	✔	메시지 종류 (§3.2)
3	`timestamp`	DateTimeOffset	✔	✔	보낸 시각
4	`clientId`	string	✔	—	**도크 ID** (예: `B22`). 응답에는 없음

3.2 `type` 값과 서버 처리

`type` 값	별칭	보내는 쪽	서버 처리	응답
`heartbeat`	—	도크	장치 상태 저장, 알람 판정, 별칭 등록	`HeartBeatResponse` (§4.1)
`processcount`	`process_count`	도크	수집 채널에 넣음	`{"status":"OK","time":...}` (§4.2)
`docksortingparam`	`dock_sorting_param`	도크	지정된 파라미터 세트 조회	`DockSortingParamResponseMessage` (§4.3)
`requesttime`	`request_time`	도크	서버 시각 응답	`{"status":"OK","time":...}`

- ⚠ **`type` 비교는 대소문자를 구분합니다.** `CanHandle` 의 `switch` 가 소문자와 정확히 일치해야 통과하므로 `"Heartbeat"` 는 거부되고 `Unknown command` 응답을 받습니다 (P8).

3.3 JSON 이 아닌 텍스트 명령 (핸들러 우선순위 순)

서버는 `ITcpMessageHandler` 구현 클래스를 모두 자동 등록하고, `Priority` 가 작은 것부터 `CanHandle` 을 호출합니다. 처음으로 `true` 를 돌려준 핸들러가 메시지를 처리합니다.

Priority	핸들러	받는 메시지	응답	비고
0	`PingCommandHandler`	`Ping` (대소문자 무시)	`Pong`	연결 확인용
1	`DockStatusUpdateMessageHandler`	`D숫자,숫자,…` (정규식 `^D\d+(,\d+)+$`)	`ACK,Dxx`	처리 코드가 주석 처리된 미완성 상태
5	`ApiDockStatusCommandHandler`	`/api/dock/status`	전체 도크 상태	⚠ `await` 가 빠져 있어 정상 응답이 아님 (P5)
10	`EchoCommandHandler`	`ECHO 문자열`	`문자열`	시험용
100	`JsonMessagedHandler`	`{` 로 시작하고 `type` 이 §3.2 값인 JSON	§4	본 프로토콜
—	(해당 핸들러 없음, 또는 응답이 비어 있음)	그 외	`Unknown command: <받은 원문>`	⚠ P4

4. TCP 메시지별 명세

4.1 `heartbeat` — 장치 상태 보고 ★

요청 (도크 → 서버) `HeartBeatReportMessage`

속성	형식	필수	설명
`messageId`, `type`, `timestamp`, `clientId`		✔	공통 머리. `type` = `"heartbeat"`, `clientId` = 도크 ID
`version`	string	—	도크 프로그램 버전 (에뮬레이터는 보내지 않음)
`deviceStatuses`	배열	✔	장치별 상태 (아래 표)
`deviceStatuses[].id`	string	✔	`scanner`, `dimensioner`, `sensor`, `conveyor`
`deviceStatuses[].status`	string	✔	상태 문자열 — 형식은 §5
`deviceStatuses[].lastSeenAt`	DateTimeOffset	✔	장치 상태를 마지막으로 읽은 시각
`dockSortingParam`	객체	—	도크가 **현재 사용 중**인 파라미터 (`dockId`, `paramName`, `updatedAt`)

- ⚠ `/api/Dock/status` 응답의 도크 ID는 `clientId`가 아니라 **`dockSortingParam.dockId` 에서 가져옵니다.** 이 속성을 빼고 보내면 화면에 `Unknown` 도크로 나타납니다 (P3).
- 에뮬레이터는 `dockSortingParam.paramName`을 빈 문자열 `""`로 보냅니다.

서버 처리 순서 (`JsonMessagedHandler` → `LogisticsSystemStateManager.GetResponseHeartbeatMessageAsync`) 1. 역직렬화합니다. `clientId`가 없으면 무시합니다. 2. 장치마다 `status`를 **4자리 hex를 쉼표로 이은 문자열**로 바꿉니다 (§5). 3. `EventDataStoreService` 메모리에 덮어씁니다(키는 `clientId`). DB에는 10초마다 기록합니다 (B15 참고). 4. 그 도크에 지정된 `DockSortingActiveParameter`를 찾습니다. 없으면 `Default` 항목을 쓰고, 이때 `dockId`는 그 도크 ID로 바꿔 넣습니다. 5. `DeviceStatusManager.ProcessHeartbeatMessageAsync`가 상태 비트 변화를 `ftms_event_log` 알람으로 남깁니다. 6. 연결 별칭을 등록합니다 (`IP:Port` ↔ 도크 ID).

응답 (서버 → 도크) `HeartBeatResponse`

속성	형식	설명
`messageId`	long	요청의 `messageId` 그대로
`type`	string	`"heartbeat"`
`timestamp`	DateTimeOffset	서버 로컬 시간
`setParameter`	객체 / 생략	서버가 **이 도크에 지시하는** 파라미터 `{dockId, paramName, updatedAt}`. 지정값과 Default가 모두 없으면 생략

- 도크는 `setParameter.paramName`이 자기가 쓰고 있는 이름과 다르면 `docksortingparam` 요청으로 세트 전체를 받아 적용하는 것이 의도된 사용법입니다. 단, 에뮬레이터에는 이 동작이 구현되어 있지 않습니다.

4.2 `processcount` — 처리수량 보고 ★

요청 (도크 → 서버) `ProcessCountMessage`

속성	형식	설명
공통 머리		`type` = `"processcount"` 또는 `"process_count"`
`processCounts`	`ProcessCountL[]`	**[0] = 이번 1건(증분)**, [1] = 도크가 누적한 값. ⚠ 서버는 **[0]만 사용**하고 [1]은 무시합니다
`rawData`	`RawData`	화물 1건의 판독 원본 (선택)

`ProcessCountL` (모든 수치는 long, 증분이면 보통 0 또는 1)

속성	하위 속성	의미
`dockId`		도크 ID
`vehicleNumber`, `route`, `pid`, `mode`		차량번호, 노선, 화물 ID, 모드(H / S) — 비어 있으면 서버가 `"-"`로 채움
`createdAt`		발생 시각
`rawdata`, `dwsdata`, `metadata`		문자열 (보통 생략) — 서버가 `rawData`를 `|`로 이은 문자열로 바꿔 `rawdata`에 채움
`scanner`	`scan`, `multi`	바코드 판독 / 다중 판독
`dimensioner`	`read_D`, `read_S`, `noread`	치수 판독 (D/S), 판독 실패
`judge`	`small`, `medium`, `large`, `irregular`	크기 판정 결과
`sorting.small` / `sorting.medium`	`auto`, `manu`	소형·중형 분류 (자동/수동)
`sorting.large`	`auto`, `manu`, `bypass`	대형 분류
`sorting.irregular`	`auto`, `manu`, `bypass`, `noRule`, `noInfo`, `trkErr`, `gapErr`	이형 분류와 오류 사유

- `Total`, `Normal`, `NG` 같은 합계는 `[JsonIgnore]` 라서 전송되지 않고, 받는 쪽에서 계산합니다.

`RawData`

속성	형식	설명
`mappedAt`	DateTimeOffset	매핑 시각
`dockId`, `vehicleNumber`, `pid`	string	⚠ 서버는 `processCounts[0].dockId` 를 **이 `dockId` 로 덮어씁니다** (P1)
`length`, `width`, `height`	int	치수 (mm)
`barcodes`	string[]	바코드 목록
`lengthAdjust`, `heightAdjust`	int	보정값
`instructed`, `judgeResult`	string	지시 슈트, 판정 결과

서버 처리 (`DataCollectorService`) 1. `processCounts[0]` 를 꺼내고 빈 필드는 `"-"` 로 채웁니다. 2. `dockId` 를 보정합니다. 먼저 `clientId` 와 다르면 `clientId` 로 바꾸고, 그 다음 `rawData.dockId` 와 다르면 **`rawData.dockId` 로 다시 바꿉니다** (보정 순서상 RawData 값이 최종값이 됨). 3. `rawData.ToString()` 결과를 `rawdata` 에 넣습니다. 형식은 `mappedAt|dockId|vehicle|pid|L|W|H|weight|bc1,bc2|lenAdj|hAdj|instructed|judge|||` 입니다. 4. 5분 누적 버퍼(메모리)에 넣고, 파일 기록 큐에도 넣습니다. 이후 DB `collected_payload` → `process_counts` → 분/시/일 집계로 이어집니다.

응답 (서버 → 도크) — 메시지를 채널에 넣는 즉시 응답합니다.

```
{"status":"OK","time":"2026-10-20T05:00:00.1234567+00:00"}
```

- `time` 은 UTC 문자열입니다(`+00:00`). ⚠ `messageId` 가 들어 있지 않아 도크는 어떤 요청에 대한 응답인지 맞춰 볼 수 없습니다 (P9).

4.3 `docksortingparam` — 분류 파라미터 세트 요청

요청: 공통 머리만 보냅니다 (`type = "docksortingparam"`).

서버 처리 1. 도크에 지정된 `DockSortingActiveParameter.paramName` 을 찾습니다. 없으면 `"Default"` 를 씁니다. 2. 그 이름의 `DockSortingParameterSet` (DB `settings`, category `param.set`)을 찾아 돌려줍니다.

응답 `DockSortingParamResponseMessage`

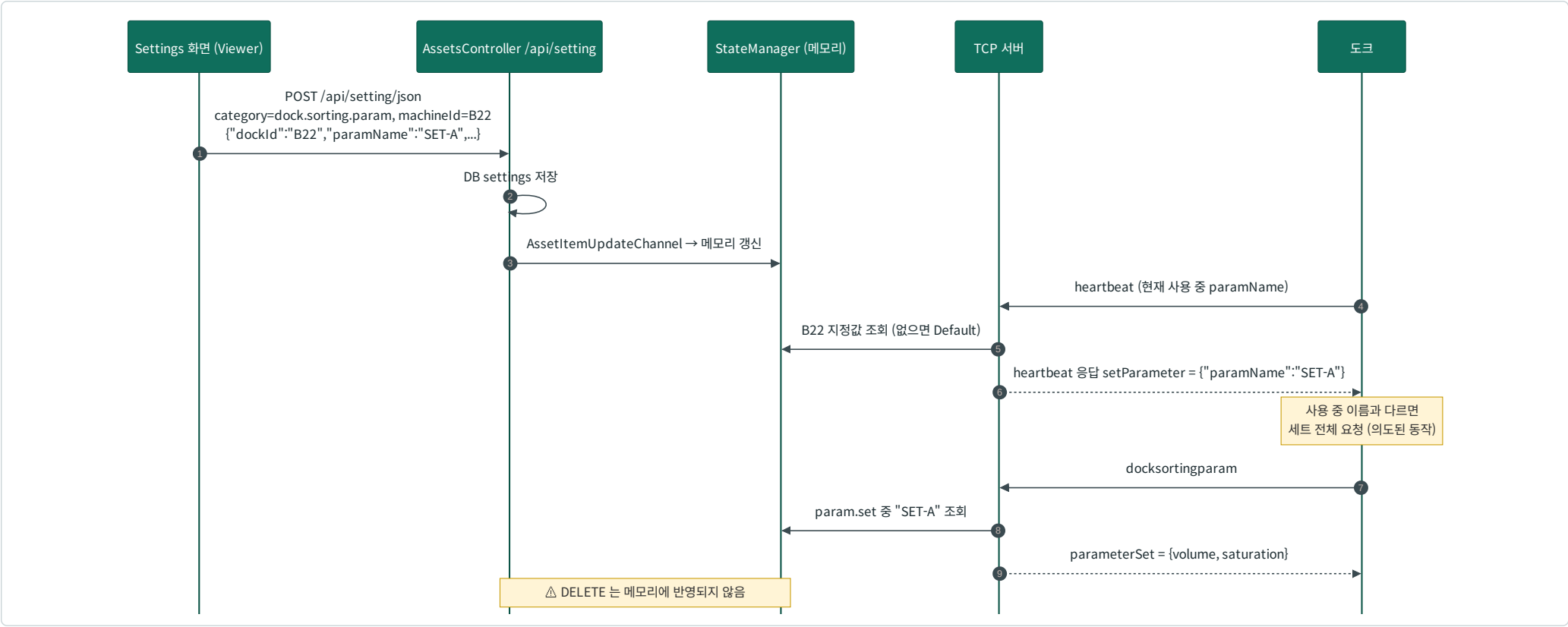
속성	설명
`messageId`	요청 번호 그대로
`type`	`"docksortingparam"`
`timestamp`	서버 시각
`parameterSet.paramName`, `.updatedAt`	세트 이름, 수정 시각
`parameterSet.volume`	`largeL/W/H`, `mediumL/W/H`, `smallL/W/H` (int, 크기 기준), `aspectLToIrregular`, `aspectSmToIrregular` (int), `useAspectLToIrregular`, `useAspectSmToIrregular` (bool)
`parameterSet.saturation`	`splitRate1` ~ `splitRate5`, `capaSmMd3Dock`, `capaLarge3Dock`, `capaIrregular3Dock` (int)

- 세트를 찾지 못하면 `parameterSet` 이 생략됩니다.
- 서버 처리 중 예외가 나면 응답이 빈 문자열이 되고, 이때는 `Unknown command: <원문>` 이 돌아갑니다 (P4).

4.4 `requesttime` — 시각 요청

- 요청: 공통 머리만 보냅니다. 응답은 `{"status":"OK","time":"<UTC ISO8601>"}` 입니다. 에뮬레이터는 이 요청을 보내지 않습니다.

그림 3. 분류 파라미터 전달 경로



- ① 설정 저장 → ③ 서버 메모리 반영 → ⑥ 다음 heartbeat 응답으로 도크에 전달됩니다.
- ⑦~⑨ 도크가 세트 전체를 요청해 받습니다. 에뮬레이터는 이 동작을 하지 않습니다 (P11).
- DELETE 는 메모리에 반영되지 않아 재시작 전까지 이전 값이 계속 전달됩니다.

5. 장치 상태 문자열 변환 규칙 ★

장치 상태는 **세 번 모양이 바뀝니다**: 도크가 보낸 문자열 → 서버가 정규화한 hex 문자열 → `/api/Dock/status` 의 숫자 배열.

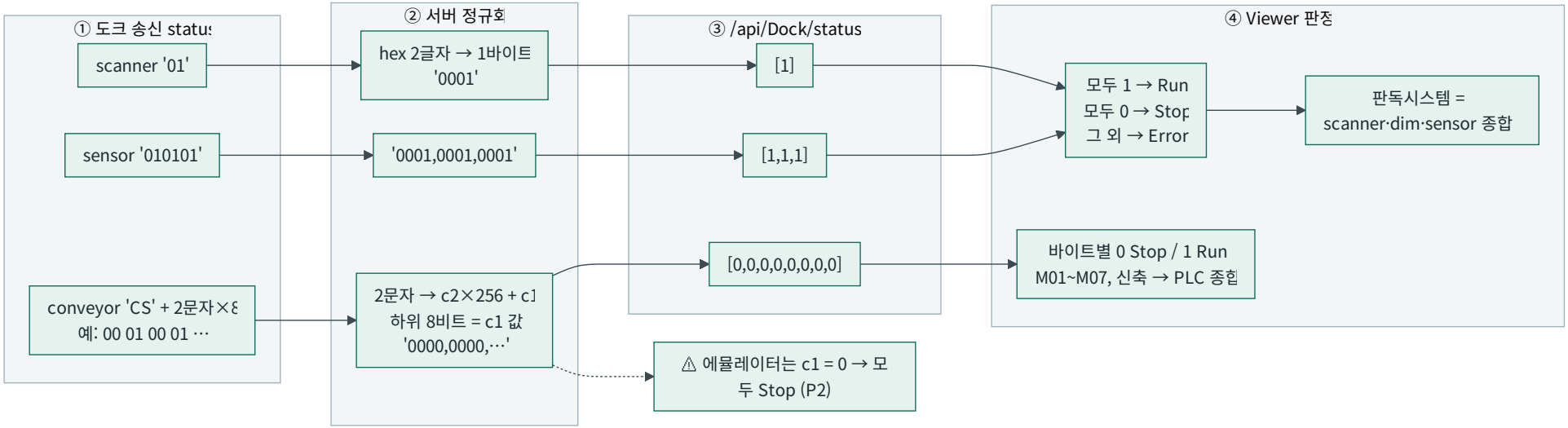
5.1 단계별 변환

단계	위치	scanner / dimensioner / sensor	conveyor
① 도크 송신	`deviceStatuses[].status`	**hex 2글자 = 1바이트** 연속. 예: `"01"`, `"010101"`	`"CS"` + **2문자 = 1워드(16bit, 리틀엔디언)** 연속. JSON 에서는 `"CS\u0001\u0000…"`
② 서버 정규화	`SanitizeStatusString_v2`	2글자씩 hex 로 파싱해 `&0xFF` → 4자리 hex 를 쉼표로 연결. 예: `"0001,0001,0001"`	`CS` 를 건너뛰고 2문자씩 `(c2<<8)|c1` 을 계산한 뒤 **`&0xFF` = 첫 문자(c1)의 값만** 사용 → `"0000,0001,…"`
③ 상태 API	`HeartBeatSerializer.ToJson` → `ExtractHexToNumbers_v2`	쉼표로 나눠 hex 를 파싱 → `[1,1,1]`	`[0,1,…]` (컨베이어 1개당 1바이트)
④ 화면 판정	`Viewer.ApplyDockStatusCompactFrame`	모두 1 → Run, 모두 0 → Stop, 그 외 → Error	바이트마다 0 → Stop, 1 → Run, 그 외 → Error

5.2 화면 판정 규칙 (Viewer)

화면 요소	판정
판독시스템 (readSystem), base, encoder	scanner, dimensioner, sensor 가 **모두 Run 이면 Run, 모두 Stop 이면 Stop, 그 외는 Error** (encoder 는 판독시스템 상태를 그대로 따라감)
컨베이어 M01~M07, 신축	conveyor 배열의 0~7번 값을 순서대로 사용. 개수가 모자라면 빠진 것은 0(Stop)
PLC 판넬	컨베이어 8개 중 Error 가 하나라도 있으면 Error, 모두 Run 이면 Run, 모두 Stop 이면 Stop, 섞여 있으면 Error
장치가 빠진 경우	해당 위치가 `null` → Error

그림 4. 장치 상태 문자열 변환 단계



- scanner·dimensioner·sensor 는 hex 2글자가 1바이트입니다.
- conveyor 는 'CS' 뒤 2문자가 1워드(리틀엔디언)이고, 하위 8비트만 사용합니다.
- 에뮬레이터 값은 하위 바이트가 0 이라 컨베이어가 모두 Stop 으로 보입니다 (P2).

5.3 ⚠ 에뮬레이터 컨베이어 값은 모두 Stop 으로 표시됨 (P2)

- 에뮬레이터는 `"CS" + "\0\u0001" × 8` 을 보냅니다. 문자 쌍 하나는 `c1='\0'`, `c2='\u0001'` 입니다.
- 서버는 `(c2<<8)|c1 = 0x0100` 을 계산하고 **`&0xFF` 를 적용해 0** 을 얻습니다. 결과적으로 컨베이어 8개가 모두 Stop, PLC 도 Stop 으로 표시됩니다.
- Run 으로 보이게 하려면 리틀엔디언 순서에 맞춰 `"CS" + "\u0001\0" × 8` 로 보내야 합니다. 실제 PLC 데이터의 바이트 순서가 어느 쪽인지 현장 확인이 필요합니다.

```
에뮬레이터 송신 :  C  S  \0 01 \0 01 \0 01 ...
서버 해석       :        [0x0100]&FF=0  [0x0100]&FF=0 ...   → Stop
Run 이 되려면   :  C  S  01 \0 01 \0 ...  → [0x0001]&FF=1 → Run
```

6. 웹 통신 (서버 ↔ Viewer)

6.1 공통

- Viewer 의 `HttpClient` 는 BaseAddress 가 **현재 사이트 주소**이므로 nginx 같은 리버스 프록시가 꼭 있어야 합니다. 모든 요청에 쿠키가 포함됩니다 (`IncludeCredentialsHandler`).
- 응답 JSON 은 ASP.NET 기본 설정(camelCase)을 따릅니다. 일부 API 는 `ApiResponse<T>` 봉투로 감싸서 돌려줍니다: `{success, statusCode, message, errorCode, traceId, elapsedMs, timestamp, affectedCount, data}`
- 대부분의 데이터 API 는 인증을 요구하지 않습니다 (S2). 전체 목록은 문서 03 (REST 부록)을 참고하세요.

6.2 Viewer 가 주기적으로 부르는 API

대상	요청	주기	응답 형식

도면	`GET /api/dxf`
	`Accept: application/x-msgpack, application/json;q=0.9`
	`If-None-Match: <ETag>`
		화면을 열 때 1 회	MessagePack + **LZ4Block** 압축 (`DxfSceneDto`), 304 이면 캐시 사용

도크 상태	`GET /api/Dock/status[?dockIds=B22&dockIds=…]`	**1초**	배열형 JSON (§6.3)
처리수량	`GET /api/ProcessCount/live/latest-n-minutes[?dockIds=…]`	**2초**	`ProcessCountL[]` (최근 5분 누적, §4.2 와 같은 구조)
알람	`POST /api/EventLog/search/fast`	**1초**	`ApiResponse<{page, pageSize, hasPreviousPage, hasNextPage, logs[]}>`
라벨·설정	`GET /api/setting?…`	화면을 열 때	설정 목록

알람 검색 요청 본문 (Viewer 기본값):

```
{"eventCode":null,"deviceId":"B22","eventLevel":"Error","deviceType":null,
 "isAcknowledged":false,"occurredFrom":"2026-10-17T05:00:00+00:00",
 "occurredTo":"2026-10-20T05:00:00+00:00","page":1,"pageSize":5}
```

- `deviceId` 는 현재 선택한 도크입니다. 선택한 도크가 없으면 `null` 이 되어 전체를 조회합니다.
- `logs[]` 의 각 항목은 `id`, `occurredAt`, `eventCode`, `deviceId`, `eventLevel`, `deviceType`, `eventDesc` 등입니다.

6.3 `/api/Dock/status` 배열형 포맷 ★

속성 이름 없이 **위치(인덱스)가 곧 의미**인 압축 형식입니다. `DockController` 가 `Ok(string)` 으로 돌려주므로 Content-Type 은 `text/plain` 이고, Viewer 는 `GetStringAsync` 로 받은 뒤 직접 파싱합니다.

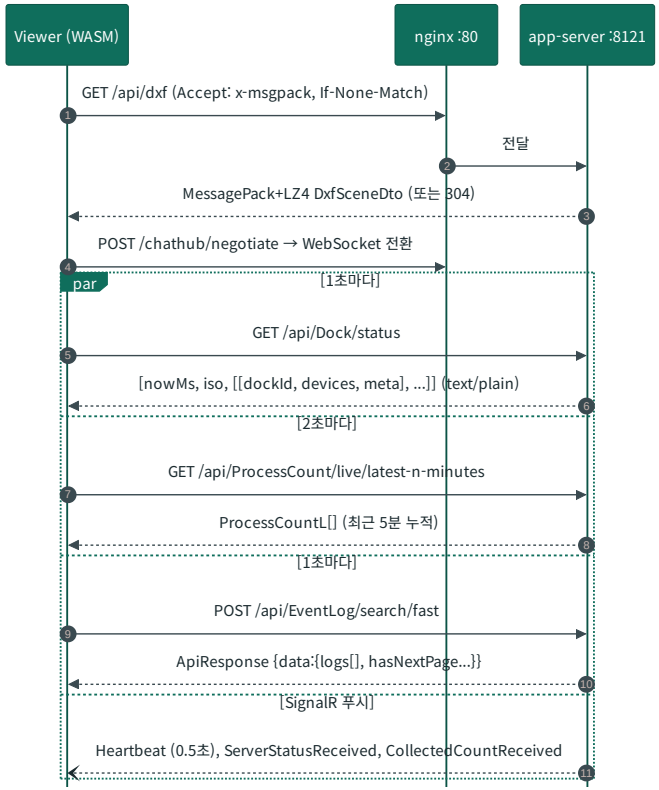
```
root = [ serverNowMs, serverNowIso, docks[] ]
docks[i] = [ dockId, devices[4], meta ]
devices = [ scanner, dimensioner, sensor, conveyor ]   ← 순서 고정, 장치가 없으면 null
device  = [ lastSeenAtMs, status int[] ]
meta    = [ dockId, paramName, updatedAtMs ] 또는 null   ← 도크가 보고한 dockSortingParam
```

인덱스	형식	의미
`root[0]`	long	서버 현재 시각 (Unix ms)
`root[1]`	string	서버 현재 시각 (ISO 8601, UTC)
`root[2]`	array	도크 목록
`dock[0]`	string	도크 ID (⚠ `dockSortingParam.dockId`, 없으면 `"Unknown"`)
`dock[1][k][0]`	long	장치 k 의 lastSeenAt (Unix ms)
`dock[1][k][1]`	int[]	장치 k 의 상태 바이트 (§5)
`dock[2]`	array / null	도크가 보고한 현재 파라미터

dock-offline 패치를 적용하면 (`setup/patches/dock-offline`) 다음 항목이 추가됩니다. 패치 전 Viewer 는 추가된 인덱스를 무시하므로 하위 호환됩니다.

인덱스	형식	의미
`root[3]`	long	끊김 판정 시간(ms) = `TcpServerFinal:TimeoutSeconds × 1000`
`dock[3]`	long	서버가 그 도크의 heartbeat 를 마지막으로 받은 시각 (Unix ms)
`dock[4]`	bool	TCP 연결 여부 (`false` 이면 화면에서 즉시 '미접속')

그림 5. Viewer ↔ 서버 통신 순서



- 도면은 처음 한 번 MessagePack+LZ4 로 받고 ETag 로 캐시합니다.
- 도크 상태 1초, 처리수량 2초, 알람 1초 주기로 폴링합니다.
- SignalR 로는 서버 Heartbeat(0.5초), 서버 상태, 수집 결과가 푸시됩니다.

6.4 SignalR `/chathub`

- 연결 URL 은 `<사이트>/chathub` 이고, 협상은 `POST /chathub/negotiate` 로 한 뒤 WebSocket 으로 전환합니다 (nginx 에 Upgrade 헤더 설정이 필요한 이유입니다).
- 끊기면 클라이언트가 **5초마다 재연결**을 시도합니다. Hub 에는 `[Authorize]` 가 없어 익명으로도 연결됩니다.
- 사용하는 프로토콜은 SignalR JSON Hub Protocol 이고, 각 메시지 끝에 구분자 `0x1E` 가 붙습니다.

방향	이벤트	인자	주기 / 계기
서버 → 전체	`Heartbeat`	`string` (서버 시각 ISO)	**0.5초** (`WebSocketHeartbeatService`)
서버 → 전체	`ServerStatusReceived`	`("ftms-app-server", ServerStatus)`	Release 약 30초, DEBUG 0.5초 (P7)
서버 → 전체	`CollectedCountReceived`	`("ftms-app-server", PayloadProcessingResultDto)`	수집 처리 결과가 바뀌었을 때
서버 → 다른 사용자	`ReceiveBroadcast`	`(보낸 사람, 메시지)`	클라이언트가 `SendBroadcast` 를 호출했을 때
서버 → 특정 사용자	`ReceivePrivateMessage`	`(보낸 사람, 메시지)`	`SendPrivateMessage`
클라이언트 → 서버	`SendBroadcast(message)` , `SendPrivateMessage(target, message)`		채팅(시험용)

`ServerStatus` = `{timestamp, serverVersion, cpuUsage, memoryUsage, dataCollectorQueueLength, tcpConnectionCount, isDbConnected, lastAggregationTime}` — ⚠ `tcpConnectionCount` 만 실제 값이고 CPU 45.5, 메모리 68.3, 큐 12, DB 연결 true 는 **고정값**입니다 (P6).

`PayloadProcessingResultDto` = `{loadedCount, dedupedCount, insertedCount, updatedCount, errorLineCount, last_N_Minutes}`

Hub 프레임 예시 (WebSocket 텍스트, 끝에 `\x1E`):

```
{"type":1,"target":"Heartbeat","arguments":["2026-10-20T05:00:00.5000000+00:00"]}\x1E
```

6.5 인증 (로그인 화면)

요청	본문 / 헤더	응답
`POST /api/auth/login`	`{"username":"…","password":"…"}`	본문: Access JWT 문자열 (15분), `Set-Cookie: ftms-api-refresh-token=…; HttpOnly` (7일)
`POST /api/auth/refresh`	쿠키	새 Access JWT. 만료 전에 자동으로 호출
`POST /api/auth/logout`	`Authorization: Bearer <JWT>`	토큰 폐기, 쿠키 삭제

7. 프로토콜 관련 문제점

번호	위치	내용	영향	권장 조치
P1	emul `Worker.ProcessCountLoop`, 서버 `DataCollectorService`	에뮬레이터는 `rawData.dockId` 를 **B22~C60 중 임의 값**으로 만들고, 서버는 `processCounts[0].dockId` 를 이 값으로 **덮어씀**	에뮬레이터 시험에서 처리수량이 **엉뚱한 도크로 집계됨**. 연결하지 않은 도크에도 수량이 생김	에뮬레이터: `RawData.DockId = id` 로 맞춤. 서버: 기준을 `clientId` 하나로 정하고 RawData 값은 경고 로그만 남김
P2	emul `HeartbeatLoop`	컨베이어 문자열의 바이트 순서가 서버 해석(리틀엔디언 하위 바이트)과 반대	에뮬레이터를 연결하면 컨베이어와 PLC 가 **항상 Stop** 으로 표시됨	`"CS" + "\u0001\0"×8` 로 수정. 실제 도크 데이터 순서를 확인
P3	`HeartBeatSerializer.SerializeDocks`	도크 ID 를 `clientId` 가 아니라 `dockSortingParam.dockId` 에서 가져옴	`dockSortingParam` 을 보내지 않는 도크는 `Unknown` 으로 표시되고, 여러 도크가 같은 ID 로 겹칠 수 있음	`report.ClientId` 를 사용
P4	`TcpServerFinalService.ProcessTcpCommandAsync`	핸들러의 응답이 비어 있으면 `Unknown command: <원문 전체>` 를 돌려보냄	heartbeat 처리가 실패하면 받은 JSON 전체를 그대로 되돌려 보냄(트래픽 낭비, 비JSON 응답)	실패 시 `{"status":"NACK","messageId":…,"error":…}` 형식으로 응답
P5	`ApiDockStatusCommandHandler`	`GetAllStatusAsync()` 에 `await` 가 없어 Task 객체를 직렬화함	TCP 로 `/api/dock/status` 를 보내도 올바른 응답이 오지 않음	`await` 를 추가하고 `ToJson()` 사용, 또는 핸들러 삭제
P6	`WebSocketHeartbeatService`	`ServerStatus` 의 CPU, 메모리, 큐, DB 연결 값이 고정값	화면의 서버 상태 표시를 믿을 수 없음	`Process`, `DataCollectorChannel` 등에서 실제 값을 읽어 채움
P7	〃	`counter++` 가 한 루프에 두 번 실행됨	상태 전송 주기가 의도한 60초가 아니라 약 30초(DEBUG 에서는 0.5초)	카운터 증가를 한 곳으로 정리
P8	`JsonMessagedHandler.CanHandle`	`type` 비교가 대소문자를 구분(`HandleAsync` 쪽은 소문자로 바꿔서 비교)	`"Heartbeat"` 처럼 대문자가 섞이면 처리되지 않음	`ToLowerInvariant()` 후 비교
P9	`processcount` 응답	응답에 `messageId` 가 없음	도크가 재전송·누락을 판단할 수 없음	응답에 `messageId` 포함
P10	`ProtocolParser` / `FillPipeAsync`	프레임 최대 길이 제한이 없음	ETX 가 빠진 비정상 데이터가 들어오면 메모리가 계속 늘어남	최대 길이(예: 1MB)를 넘으면 버퍼를 버리고 연결 종료
P11	에뮬레이터	응답(`setParameter`, `parameterSet`)을 해석하지 않음	설정 변경이 도크까지 전달되는지 에뮬레이터로는 확인할 수 없음	로그로라도 파라미터 변경을 표시
P12	타임아웃 값	서버 10초, 에뮬레이터 60초로 서로 다름	서버가 끊은 연결을 에뮬레이터는 쓰기에 실패하거나 60초가 지나서야 알아차림	양쪽 설정을 맞추고, 에뮬레이터는 보내기 실패 시 즉시 재연결

8. 직접 확인하는 방법

8.1 TCP 를 손으로 보내 보기 (PowerShell)

PowerShell 에서 아래 명령을 한 줄씩 실행합니다. 서버(app-server.ftms)가 실행 중이어야 합니다.

```
$c = New-Object Net.Sockets.TcpClient('127.0.0.1', 6004)
$s = $c.GetStream()
$json = '{"messageId":1,"type":"heartbeat","timestamp":"2026-10-20T14:00:00+09:00","clientId":"T01","deviceStatuses":
[{"id":"scanner","status":"01","lastSeenAt":"2026-10-20T14:00:00+09:00"}],"dockSortingParam":{"dockId":"T01","paramName":"","updatedAt":"2026-
10-20T14:00:00+09:00"}}'
$b = [Text.Encoding]::UTF8.GetBytes([char]2 + $json + [char]3)
$s.Write($b, 0, $b.Length)
Start-Sleep -Milliseconds 500
$buf = New-Object byte[] 4096; $n = $s.Read($buf, 0, 4096)
[Text.Encoding]::UTF8.GetString($buf, 1, $n - 2)
$c.Close()
```

- 정상이라면 `{"messageId":1,"type":"heartbeat","timestamp":"…","setParameter":{…}}` 가 출력됩니다(Default 파라미터가 없으면 `setParameter` 는 생략).
- 10초 안에 다음 데이터를 보내지 않으면 서버가 연결을 끊습니다.

8.2 상태 API 확인

```
curl.exe -s http://localhost/api/Dock/status
```

- `[1760936400123,"2026-…",[["T01",[[…,[1]],null,null,null],["T01","",…]]]]` 처럼 §6.3 형식으로 나오면 정상입니다.

8.3 서버 로그에서 송수신 보기

- 서버 콘솔에 `Received from <clientId> (N bytes): {…}` 와 `Sent to <clientId> …` 가 찍힙니다. 기본 로그 레벨(Information)에서 보입니다.
- 연결이 많아 로그가 너무 많으면 클라이언트별로 레벨을 낮출 수 있습니다: `PUT /api/RuntimeConfigChange/clients/by-nickname/log-level`

부록. 메시지 예제 모음

A. heartbeat 요청 (에뮬레이터 B22, 실제 직렬화 형태)

```
{"messageId":101,"type":"heartbeat","timestamp":"2026-10-20T14:00:01.0000000+09:00","clientId":"B22",
 "deviceStatuses":[
  {"id":"scanner","status":"01","lastSeenAt":"2026-10-20T14:00:01.0000000+09:00"},
  {"id":"dimensioner","status":"01","lastSeenAt":"2026-10-20T14:00:01.0000000+09:00"},
  {"id":"sensor","status":"010101","lastSeenAt":"2026-10-20T14:00:01.0000000+09:00"},

{"id":"conveyor","status":"CS\u0000\u0001\u0000\u0001\u0000\u0001\u0000\u0001\u0000\u0001\u0000\u0001\u0000\u0001\u0000\u0001","lastSeenAt":"2026-
10-20T14:00:01.0000000+09:00"}],
 "dockSortingParam":{"dockId":"B22","paramName":"","updatedAt":"2026-10-20T14:00:01.0000000+09:00"}}
```

(실제로는 줄바꿈 없이 한 줄이고, 앞에 0x02, 뒤에 0x03 이 붙습니다.)

B. heartbeat 응답

```
{"messageId":101,"type":"heartbeat","timestamp":"2026-10-20T14:00:01.0100000+09:00",
 "setParameter":{"dockId":"B22","paramName":"Default","updatedAt":"2026-10-19T09:00:00.0000000+09:00"}}
```

C. 서버 메모리에 저장되는 정규화 결과 (A 를 받은 뒤)

장치	받은 값	저장 값	[illegible]
scanner	`01`	`0001`	`[1]` → Run
dimensioner	`01`	`0001`	`[1]` → Run
sensor	`010101`	`0001,0001,0001`	`[1,1,1]` → Run
conveyor	`CS\0\1…`	`0000,0000,0000,0000,0000,0000,0000,0000`	`[0,0,0,0,0,0,0,0]` → 모두 Stop (P2)

D. processcount 요청

```
{"messageId":2051,"type":"processcount","timestamp":"2026-10-20T14:00:01.2000000+09:00","clientId":"B22",
 "processCounts":[
  {"dockId":"B22","vehicleNumber":"VIN-7","route":"Route-2","pid":"PID-4821","mode":"H","createdAt":"2026-10-20T14:00:01.2000000+09:00",
   "scanner":{"scan":1,"multi":0},"dimensioner":{"read_D":0,"read_S":1,"noread":0},
   "judge":{"small":0,"medium":1,"large":0,"irregular":0},
   "sorting":{"small":{"auto":0,"manu":0},"medium":{"auto":1,"manu":0},"large":{"auto":0,"manu":0,"bypass":0},
              "irregular":{"auto":0,"manu":0,"bypass":0,"noRule":0,"noInfo":0,"trkErr":0,"gapErr":0}}},
  {"dockId":"B22","…":"누적값(서버는 사용 안 함)"}],
 "rawData":{"mappedAt":"2026-10-20T14:00:01.2000000+09:00","dockId":"C41","vehicleNumber":"VIN-3","pid":"PID-1290",
   "length":512,"width":300,"height":250,"barcodes":["BC-1001","BC-2002","BC-3003"],
   "lengthAdjust":-12,"heightAdjust":30,"instructed":"Instructed-2","judgeResult":"JudgeResult-1"}}
```

→ `rawData.dockId` 가 `C41` 이라서 서버는 이 1건을 **C41 도크의 수량**으로 기록합니다 (P1).

E. processcount 응답

```
{"status":"OK","time":"2026-10-20T05:00:01.2050000+00:00"}
```

F. docksortingparam 요청 / 응답

```
{"messageId":3001,"type":"docksortingparam","timestamp":"2026-10-20T14:01:00.0000000+09:00","clientId":"B22"}
```

```
{"messageId":3001,"type":"docksortingparam","timestamp":"2026-10-20T14:01:00.0100000+09:00",
 "parameterSet":{"paramName":"Default","updatedAt":"2026-10-19T09:00:00.0000000+09:00",
  "volume":{"largeL":800,"largeW":600,"largeH":600,"mediumL":500,"mediumW":400,"mediumH":400,"smallL":300,"smallW":200,"smallH":200,
            "aspectLToIrregular":5,"aspectSmToIrregular":4,"useAspectLToIrregular":true,"useAspectSmToIrregular":false},
  "saturation":{"splitRate1":20,"splitRate2":20,"splitRate3":20,"splitRate4":20,"splitRate5":20,
               "capaSmMd3Dock":100,"capaLarge3Dock":50,"capaIrregular3Dock":30}}}
```

(수치는 예시입니다. 실제 값은 DB `settings` 의 `param.set` 항목에 따릅니다.)

G. `/api/Dock/status` 응답 (도크 2개)


```
[1760936401123,"2026-10-20T05:00:01.1230000+00:00",[
 ["B22",[[1760936401000,[1]],[1760936401000,[1]],[1760936401000,[1,1,1]],[1760936401000,[0,0,0,0,0,0,0,0]]],["B22","",1760936401000]],
 ["B23",[[1760936400990,[1]],[1760936400990,[1]],[1760936400990,[1,1,1]],[1760936400990,[0,0,0,0,0,0,0,0]]],["B23","",17609364009990]]
]]
```